

Session 4 Speaker Script — COMPLETE WITH FULL TALKING POINTS

# Structured Outputs & Parsing

**Presenter:** Dinesh Wadhvani | **Session:** 4 of 12 | **Duration:** ~60 minutes

**Purpose:** Connect AI outputs to real systems via structured data

| Slide | Title                            | Time    |
|-------|----------------------------------|---------|
| 1     | Title Slide                      | ~1 min  |
| 2     | Today's Journey                  | ~2 min  |
| 3     | Why Structured Outputs?          | ~5 min  |
| 4     | JSON: The Language of Data       | ~8 min  |
| 5     | Other Formats: CSV & XML         | ~5 min  |
| 6     | Prompting for Structured Output  | ~8 min  |
| 7     | Validation & Error Handling      | ~8 min  |
| 8     | Real-World: AI Agents & Products | ~8 min  |
| 9     | Building AI Pipelines            | ~5 min  |
| 10    | Lab Overview                     | ~3 min  |
| 11    | Key Takeaways                    | ~5 min  |
| 12    | Session 5 Preview                | ~2 min  |
| —     | Lab Exercise                     | ~12 min |

## 01. TITLE SLIDE

~1 minute

### **WHAT'S ON SCREEN**

Session 4 label (blue) • Title: 'Structured Outputs & Parsing' • Subtitle: 'Connecting AI to Real Systems' • Session 4 of 12  
| 60 minutes

### **SPEAKING SCRIPT**

Welcome to Session 4. We've covered a lot of ground. Session 1: you learned what LLMs are and how they fit into the AI landscape. Session 2: you opened the hood and understood the mechanics—tokens, transformers, attention. Session 3: you mastered prompt engineering, learning to write prompts that reliably produce better outputs.

Today: we take it to the next level. Everything so far has produced text output. Today we're adding a crucial capability: structured output. The ability to make LLMs generate data in formats that other systems can consume automatically.

This is where AI stops being just 'a smart chatbot that writes text' and becomes 'a system that feeds into real workflows.' When an LLM generates JSON that a database can ingest, or CSV that a spreadsheet can process, or structured data that a third system can parse—that's when LLMs become infrastructure.

By the end of today, you'll understand how to design prompts that output structured data, how to validate that data, and how to build simple pipelines where AI connects to other systems. You'll see real products doing this: GitHub Copilot, customer service platforms, booking systems—all using structured outputs under the hood.

Let's begin.

### **DELIVERY TIPS**

High energy—this is where AI becomes practical infrastructure. • Emphasize the shift: text output → structured data → real systems. • Quick context: 'Anyone used GitHub Copilot? Or a resume parser? Those use structured outputs.'

## 02. TODAY'S JOURNEY

~2 minutes

### **WHAT'S ON SCREEN**

12-item agenda • Items: Why Structured?, JSON, CSV/XML, Prompting, Validation, Real Examples, Pipelines, Lab, Takeaways, Session 5 Preview

### **SPEAKING SCRIPT**

Quick roadmap. We start with a five-minute exploration: why does structured output matter? Why not just use text? What problem does it solve?

Then eight minutes on JSON. JSON is the lingua franca of structured data on the web. Every API, every modern system speaks JSON. We'll learn what it is, why it matters, and see it in context.

Five minutes on CSV and XML—other formats you'll encounter. When to use each.

Eight minutes on prompting for structured output. It's different from prompting for text. You have to be precise about the exact schema—the shape of the data.

Eight minutes on validation and error handling. What happens when the LLM generates invalid JSON? How do you handle it? What strategies work?

Eight minutes on real-world examples. I'll show you actual AI agents and products that use structured outputs. GitHub Copilot, customer service bots, booking systems, content platforms. You'll see the pattern.

Five minutes on building pipelines. How do you string together: AI generates data → validate it → feed it to another system → process it?

Then lab—12 minutes. You'll take your resume prompt from Session 3, modify it to output JSON, validate the JSON, and build a simple pipeline that extracts data from it.

Then takeaways and Session 5 preview. If we get tight on time, we compress the additional formats section. We protect the real-world examples and lab.

### **DELIVERY TIPS**

Move briskly—this is a lot of ground. • Emphasize that students will connect AI to real systems today. • Preview the lab: 'By the end, you'll have built something you can actually use.'

## 03. WHY STRUCTURED OUTPUTS?

~5 minutes

### WHAT'S ON SCREEN

Problem-Solution visual • Left: 'Unstructured text' problem, Right: 'Structured data' solution • Real examples of each • Business impact metrics

### SPEAKING SCRIPT

Let me start with a problem. Imagine you're a hiring company. You have 10,000 resumes submitted. You want to use AI to extract: candidate name, years of experience, top skills, previous companies.

PROBLEM: If the LLM outputs regular text like 'The candidate John Smith has 5 years of experience. Skills include Python, data analysis, and cloud computing. Previously worked at Google and Amazon.' What do you do? A human reads it fine. But how does your system automatically process it? How does it feed that into your database? How does it query it later?

You'd have to: write custom parsing code, hope the AI formats it consistently (it won't), handle edge cases when the LLM formats it differently. Fragile. Error-prone. Expensive.

SOLUTION: If the LLM outputs structured JSON: `{"name": "John Smith", "experience_years": 5, "skills": ["Python", "Data Analysis", "Cloud Computing"], "previous_companies": ["Google", "Amazon"]}` Now your system can: automatically read the JSON, validate it, insert it into database, query it, process it. No custom parsing. No ambiguity. Clean.

This is the difference between AI-as-entertainment and AI-as-infrastructure.

### REAL-WORLD IMPACT:

LinkedIn uses AI to parse resumes into structured data. Why? Because when you upload a resume, they need to extract your work history, education, skills—and populate your profile automatically. They can't do that with unstructured text. They need structured data.

GitHub Copilot, when you ask it to generate code, produces structured Python/JavaScript/whatever. Not because it's being creative. But because downstream, your IDE has to parse it, syntax-highlight it, run it. Your IDE needs structure, not random text.

Customer service chatbots that extract your order number, issue description, and priority level—they do this by prompting the AI for structured output. Then their system automatically routes your ticket. No human interpretation needed.

Zapier and Make (automation platforms) use AI to generate structured workflows. They prompt AI: 'Generate a workflow JSON that: when Gmail receives email from X, extract attachment, upload to Google Drive.' The output is structured so it can be directly executed.

### BOTTOM LINE:

Unstructured text is for humans. Structured data is for systems. When you want LLMs to be part of real workflows, you need structured output.

### DELIVERY TIPS

Use the hiring company example—it's relatable and concrete. • The LinkedIn/GitHub/Zapier examples show this is real and widespread. • Emphasize the shift: AI stops being 'smart chat' and becomes 'data source for systems.' • The JSON example is key—show the contrast visually if possible.

## 04. JSON: THE LANGUAGE OF DATA

~8 minutes

### WHAT'S ON SCREEN

JSON structure visual • Key-value pairs shown • Nested objects and arrays • Real resume example in JSON format • Comparison: unstructured vs structured

### SPEAKING SCRIPT

JSON stands for JavaScript Object Notation. Despite the name, it's not just for JavaScript. It's the de facto standard for structured data on the web. Every API uses it. Every modern system understands it.

WHAT IS JSON?

JSON is a way to organize data into a structure. It has simple rules: data is organized as key-value pairs. Keys are labels. Values are the actual data.

Simple example: `{"name": "Priya", "age": 22, "major": "Computer Science"}`

Keys: name, age, major. Values: Priya, 22, Computer Science.

WHY JSON?

Because it's: simple (easy to read and understand), standard (everyone speaks it), machine-readable (systems can parse it automatically), flexible (you can nest data), and widely supported (every programming language has JSON libraries).

STRUCTURE BASICS:

1. OBJECTS: Enclosed in curly braces `{}`. Contains key-value pairs.

Example: `{"first_name": "John", "last_name": "Smith"}`

2. ARRAYS: Enclosed in square brackets `[]`. Contains a list of values.

Example: `{"skills": ["Python", "SQL", "AWS"]}`

3. NESTING: You can put objects inside objects.

Example: `{"candidate": {"name": "John", "address": {"city": "New York", "state": "NY"}}}`

4. DATA TYPES: JSON supports strings (text), numbers, booleans (true/false), null (empty), arrays, and objects.

REAL-WORLD EXAMPLE: RESUME IN JSON

Imagine a resume. In text form, it's messy: names, dates, bullet points, inconsistent formatting. In JSON, it's clean:

```
{
  "candidate": {
    "name": "Priya Sharma",
    "email": "priya@email.com",
    "phone": "555-1234"
  },
  "experience": [
    {
      "company": "TechCorp",
      "position": "Data Analyst",
      "duration": "2 years",
      "achievements": ["Built dashboards", "Optimized queries"]
    },
    {
      "company": "StartupXYZ",
      "position": "Intern",
```

```
"duration": "3 months",
"achievements": ["Learned cloud infrastructure"]
},
"skills": ["Python", "SQL", "Tableau"],
"education": {
  "degree": "B.Tech",
  "field": "Computer Science",
  "university": "University Name",
  "year": 2024
}
}
```

Now notice: every piece of data has a label. The structure is clear. A system can immediately extract: what companies did this person work for? (Look in experience array, get company field). What skills do they have? (Look in skills array). When did they graduate? (Look in education.year).

#### WHY THIS MATTERS FOR PROMPTING:

When you prompt an LLM for structured output, you typically say: 'Generate output in this JSON format: [you specify the schema]. The schema is the blueprint—the shape of the data you want.

By being explicit about the schema, you're telling the model: 'Here's exactly what I want. These are the fields. These are the data types. Follow this structure.'

The model can then generate output that matches your schema exactly. No ambiguity. No parsing needed. Your system reads it directly.

#### **DELIVERY TIPS**

JSON feels technical but it's actually simple. Make it approachable. • The resume example is concrete—students can visualize this. • Show the contrast: messy text resume vs clean JSON resume. • Emphasize: you don't need to write JSON from scratch. You specify the schema and the LLM generates it. • Connection: 'Just like in Session 3 when you specified output format, here you're specifying the exact data structure.'

## 05. OTHER FORMATS: CSV & XML

~5 minutes

### WHAT'S ON SCREEN

Three format comparison table • JSON vs CSV vs XML • When to use each • Real-world use cases for each

### SPEAKING SCRIPT

JSON is most common, but you'll encounter other formats. Let me quickly cover CSV and XML.

CSV (COMMA-SEPARATED VALUES):

CSV is simple: data in rows and columns, separated by commas. It's what you get when you export a spreadsheet.

Example:

```
name, years_experience, top_skill
Priya, 2, Python
John, 5, Java
Maria, 3, SQL
```

When to use: When the data is naturally tabular (rows and columns). When it needs to open in Excel or Google Sheets. When the data structure is flat (not nested).

Downside: CSV doesn't handle nested data well. It's simple but limited.

XML (EXTENSIBLE MARKUP LANGUAGE):

XML is verbose but powerful. It uses tags like HTML to structure data.

Example:

```
<candidate>
<name>Priya Sharma</name>
<experience>
<job>
<company>TechCorp</company>
<position>Data Analyst</position>
</job>
</experience>
</candidate>
```

When to use: When data is hierarchical (nested). When you need validation rules. In enterprise systems (banks, healthcare often use XML). When you need to represent complex relationships.

Downside: Verbose (takes more space). Less common in modern web development (JSON replaced it).

COMPARISON:

JSON: Modern, concise, flexible, standard for web APIs. Use this if you're starting today.

CSV: Simple, works in spreadsheets, flat data only. Use when data is tabular and you need Excel compatibility.

XML: Complex, hierarchical, verbose. Use in enterprise systems or when you need strict validation.

IN TODAY'S LAB:

We're using JSON because it's the modern standard. But the same principles apply to CSV or XML. You specify the format in your prompt. The model generates it. You validate it. You process it.

### DELIVERY TIPS

Keep this section brief—JSON is the focus. • CSV and XML are just for awareness. • The comparison table helps students understand trade-offs. • Emphasize: JSON is the safe default choice for new projects.



## 06. PROMPTING FOR STRUCTURED OUTPUT

~8 minutes

### WHAT'S ON SCREEN

Prompt structure template • Schema definition shown • Before/After example • Key elements highlighted: format, schema, constraints

### SPEAKING SCRIPT

Here's where Session 3 techniques meet Session 4 requirements. You need to prompt for structured output in a way that reliably produces valid data.

#### THE DIFFERENCE FROM TEXT PROMPTING:

In Session 3, you said: 'Write a resume that is professional, well-formatted, includes key sections.' Vague but workable for text.

For structured output, you need to be EXACT. You're not saying 'include key sections.' You're saying: 'Output JSON with these exact fields: name (string), experience\_years (number), skills (array of strings), etc.'

#### WHY EXACTNESS MATTERS:

Because your system is going to parse this JSON. If the structure is wrong—maybe a field name is different, or the data type is wrong—your parsing code breaks. With text, a human can figure out what you meant. With structured data, the machine can't. It's rigorous.

#### TEMPLATE FOR PROMPTING STRUCTURED OUTPUT:

Use this structure:

---START TEMPLATE---

You are a [ROLE]. Generate structured data in JSON format. The JSON must have this exact schema:

```
{
  "field_1": [data type and description],
  "field_2": [data type and description],
  "field_3_array": ["array of", "type", "with description"]
}
```

Important constraints:

- Field names must match exactly (case-sensitive)
- Data types: use strings for text, numbers for quantities, booleans for true/false
- Dates in ISO format (YYYY-MM-DD)
- Always include all fields, even if null
- Generate the JSON, nothing else

[INPUT DATA]■r/>---END TEMPLATE---

REAL EXAMPLE: Extract resume data

"You are a resume parser. Extract data from this resume and output as JSON with this exact schema:

```
{
  \"name\": \"string (first and last name)\",
  \"years_experience\": number (total years),
  \"top_skills\": [\"array\", \"of\", \"strings\"],
  \"current_company\": \"string or null if not specified\",
  \"education_degree\": \"string (e.g. B.Tech, MBA)\",
  \"graduate_year\": number (YYYY format)
}
```

Requirements:

- Names must be exactly as written in resume
- Years of experience: sum all positions, round to nearest year
- Skills: extract only if explicitly mentioned, maximum 5
- Company: null if current employment not specified
- Dates in YYYY format only
- Output ONLY the JSON, no explanation

[Resume text here]"

CRITICAL ELEMENTS:

1. SCHEMA SPECIFICATION: You must define every field, its data type, and what it contains. The more explicit, the better.
2. CONSTRAINTS: Rules about formatting, validation, data types. Things like 'dates in ISO format', 'round to nearest integer', 'must include all fields.'
3. OUTPUT FORMAT: Say 'output ONLY JSON' so the model doesn't add explanation text before or after.
4. EXAMPLES (Optional but powerful): Show 1-2 examples of correct JSON format. Few-shot learning for structured data.

WHY THIS WORKS:

Remember from Session 2: the model's attention mechanism focuses on what matters in your prompt. By being explicit about schema and constraints, you activate the right patterns. The model learns: 'I need to generate JSON matching this exact structure.'

COMMON MISTAKES (What NOT to do):

- Vague: 'Generate resume data in JSON format' (what fields? what structure?)
- Explicit: 'Generate JSON with fields: name (string), years\_experience (number), skills (array), etc.'
- Forgetting constraints: Model might use wrong data types
- Explicit: 'years\_experience must be a number, not a string like "5 years"'
- Allowing extra text: 'Here's the JSON: [JSON here]' (extra text breaks parsing)
- Clear: 'Output ONLY valid JSON, no other text'

IN TODAY'S LAB:

You'll take your resume prompt from Session 3, add schema and constraints, and get JSON output. Then validate it. Then process it.

### ***DELIVERY TIPS***

The template is practical—students will use it immediately. • The resume example is concrete and familiar (they've been working with resumes all course). • Emphasize: being explicit about schema reduces errors dramatically. • Show what 'wrong' looks like and 'right' looks like—contrast is powerful. • Connection back: 'Remember Session 3 when you specified output format? This is output format on steroids.'

## 07. VALIDATION & ERROR HANDLING

~8 minutes

### WHAT'S ON SCREEN

Error scenarios visual • Invalid JSON examples shown • Validation strategies listed • Recovery approaches highlighted

### SPEAKING SCRIPT

Here's something crucial that people skip: what happens when the LLM generates invalid JSON? Because it will happen. It's not if, it's when.

#### WHY VALIDATION MATTERS:

You prompt the model, it generates JSON that's supposed to match your schema. But then: a field name is misspelled. A number is in quotes (string instead of number). A required field is missing. The JSON is malformed.

If you skip validation and try to parse it, your system crashes. Or it silently inserts garbage data into your database. Neither is good.

#### TYPES OF ERRORS:

1. MALFORMED JSON: The JSON syntax is broken. Not valid JSON at all.

Example: `{"name": "John", "age": 30,}` (trailing comma—invalid)

Your system: Can't parse it at all. Error.

2. SCHEMA MISMATCH: JSON is valid but doesn't match your schema.

Example: Your schema says `{"experience_years": number}` but model generates `{"experience_years": "5 years"}` (string instead of number)

Your system: Can parse it, but then fails when it tries to use it as a number.

3. MISSING FIELDS: JSON is valid but is missing required fields.

Example: Your schema needs name, email, phone. Model only generates name and email. Missing phone.

Your system: Can parse, but incomplete data breaks downstream processing.

4. UNEXPECTED EXTRA FIELDS: JSON has fields you didn't ask for.

Usually harmless but can indicate the model misunderstood the schema.

5. WRONG DATA TYPES: Field is present but wrong type.

Example: Expected `"age": 30` but got `"age": true` (boolean instead of number)

#### VALIDATION STRATEGIES:

##### STRATEGY 1: TRY-CATCH (Basic)

Attempt to parse the JSON. If it fails, log the error and ask for a retry.

Pseudo-code: `try { parse JSON } catch { error—ask model to regenerate }`

Downside: Only catches malformed JSON, not schema mismatches.

##### STRATEGY 2: SCHEMA VALIDATION (Better)

Parse the JSON, then check: does every required field exist? Are the data types correct? Do values match expected ranges?

Pseudo-code: `parse JSON, then validate each field against schema`

Downside: Requires setting up a validator, but worth it for production systems.

##### STRATEGY 3: ITERATIVE REFINEMENT (Best)

If validation fails, tell the model specifically what's wrong and ask it to regenerate.

Example: 'Your output was invalid. The field "experience\_years" should be a number, not a string. Please regenerate the JSON.'

Downside: Takes more time, more API calls, but produces reliable output eventually.

REAL-WORLD EXAMPLE: GitHub Copilot

When you ask Copilot to generate code, it produces code strings. GitHub then: 1) Checks syntax (is it valid Python/JavaScript/whatever?), 2) Validates it matches expected patterns, 3) If invalid, asks the model to regenerate.

When the error rate gets too high on a specific request, they might change the prompt, add examples, or refine constraints.

LINKEDIN Resume Parser:

When you upload a resume, LinkedIn uses AI to extract data. They validate: 1) All required fields extracted, 2) Data types correct, 3) Values make sense (experience years is a positive number, not negative). If validation fails on critical fields, the system falls back to a human review.

ERROR RECOVERY PATTERNS:

Pattern 1: Strict Validation + Retry

If invalid, immediately ask model to regenerate. Good for simple structures.

Pattern 2: Partial Recovery + Human Review

If most fields valid but some bad, use valid parts, flag bad parts for human. Good for mixed scenarios.

Pattern 3: Graceful Degradation

If complete JSON invalid, extract what's usable, use defaults for missing. Good for non-critical data.

IN TODAY'S LAB:

You'll generate JSON, validate it (we'll provide a simple validator), see if it passes, and if it doesn't, you'll iterate on the prompt to fix it. This is professional-level workflow design.

### ***DELIVERY TIPS***

Validation feels technical but it's essential for real systems. Emphasize this isn't optional. • Real-world examples (GitHub, LinkedIn) show this is how it's done. • The error scenarios are concrete—students will recognize these. • Iterative refinement pattern is powerful—show that you can ask the model 'you made this error, fix it.' • Connection: 'This is why prompt engineering from Session 3 matters—better prompts = fewer validation errors.'

## 08. REAL-WORLD: AI AGENTS & PRODUCTS

~8 minutes

### WHAT'S ON SCREEN

5 Real products/agents shown • GitHub Copilot, ChatGPT Code Interpreter, Customer Service Bots, Resume Parsers, Zapier/Make • How each uses structured outputs • Business impact for each

### SPEAKING SCRIPT

Let me show you how real AI agents and products use structured outputs. This isn't theoretical. This is how billion-dollar companies work today.

#### PRODUCT 1: GITHUB COPILOT

What it does: You describe what code you want ('a function that sorts a list'). Copilot generates code.■r/>

Behind the scenes (structured output):

- 1) Your request goes to an LLM
- 2) LLM generates code as a STRING■r/>3) GitHub validates it: 'Is this valid Python/JavaScript/whatever syntax?'■r/>4) If valid, syntax-highlights it and shows it to you■r/>5) You can accept it, modify it, or reject it■r/>6) If you accept, it's inserted into your IDE■r/>

Why structured output matters: The code HAS to be valid syntax. If Copilot generated random text that looked like code but wasn't, it would be useless. GitHub uses validation to ensure what's generated is actually executable.

Real impact: GitHub reports Copilot generates code 40% faster, with fewer syntax errors than manual coding. That's because the prompts are precisely engineered, and structured validation ensures quality.■r/>

#### PRODUCT 2: CHATGPT CODE INTERPRETER

What it does: You ask ChatGPT a question with code ('analyze this dataset'). ChatGPT doesn't just explain—it writes code, GENERATES a JSON request to run it, then actually EXECUTES the code and shows you results.

Behind the scenes (structured output):

- 1) You ask a question■r/>2) ChatGPT generates Python code as a STRING■r/>3) ChatGPT then generates a JSON REQUEST with schema like {"action": "execute\_code", "code": "[python code]", "parameters": {...}}■r/>4) The system validates this JSON■r/>5) If valid, it executes the code■r/>6) Code produces output■r/>7) That output is fed back to ChatGPT, which explains results to you

Why structured output matters: The system needs to parse the JSON to know: what action to take, what code to run, what parameters to use. Without structured output, ChatGPT couldn't trigger the code interpreter.■r/>

Real impact: You ask a question and get not just explanation but analysis + code + results. Impossible without structured outputs.

#### PRODUCT 3: CUSTOMER SERVICE CHATBOTS (Zendesk, Intercom)

What it does: Customer sends a support message. AI chatbot reads it, extracts the issue, and routes your ticket to the right team.■r/>

Behind the scenes (structured output):

- 1) Customer writes: 'I ordered item X yesterday and it hasn't arrived. Order number 12345. Very frustrated.'■r/>2) AI generates structured JSON like: {"issue\_type": "delayed\_delivery", "order\_id": 12345, "sentiment": "negative", "urgency": "high", "suggested\_team": "fulfillment"}■r/>3) System validates the JSON■r/>4) System automatically routes to fulfillment team■r/>5) Fulfillment team sees the structured data: order ID, issue type, urgency—everything they need

Why structured output matters: Routing has to be automatic. The company processes thousands of tickets per hour. Can't have humans reading each one. Need structured data so systems can route automatically.

Real impact: Instead of 5 minutes per ticket to triage (human), system processes in milliseconds. Cost savings, customer satisfaction up.

#### PRODUCT 4: LINKEDIN RESUME PARSER

What it does: You upload a resume PDF. LinkedIn automatically extracts your work history, skills, education, and

populates your profile.

Behind the scenes (structured output):

1) Resume PDF is converted to text  
2) LLM is prompted: 'Extract resume data into this JSON schema: {name, email, work\_history[], skills[], education}'  
3) LLM generates JSON  
4) LinkedIn validates it: 'All required fields? Correct data types?'  
5) LinkedIn inserts into database  
6) Your profile is auto-populated

Why structured output matters: They process millions of resumes. Can't have humans manually entering data. Need structured extraction so it can go directly to database.

Real impact: Users spend 30 seconds uploading resume instead of 15 minutes manually entering data. LinkedIn builds better user profiles. Win-win.

#### PRODUCT 5: ZAPIER / MAKE (Automation Platforms)

What it does: You set up workflows like 'When an email arrives from my boss, create a task in my to-do app.' Increasingly, Zapier uses AI to help create these workflows.

Behind the scenes (structured output):

1) User says (in plain English): 'When I get an email with a high priority, create a task in Asana'  
2) AI generates a structured workflow JSON like: {"trigger": {"service": "gmail", "event": "email\_received", "filter": "priority=high"}, "action": {"service": "asana", "event": "create\_task", "fields": {"title": "from\_email\_subject", "description": "from\_email\_body"}}}  
3) Zapier validates this JSON against their workflow schema  
4) If valid, it creates the workflow  
5) From then on, whenever you get a high-priority email, a task is automatically created in Asana

Why structured output matters: The workflow HAS to be executable. Zapier needs to know exactly: which service to connect, what event to trigger, what actions to take, what data to map. Needs structured JSON for this.

Real impact: Non-technical users can now create complex workflows. Millions of workflows automated. Revenue unlocked.

#### COMMON PATTERN ACROSS ALL FIVE:

Notice what's consistent: In each case, the LLM generates output → that output is structured (code, JSON, schema) → system validates it → system processes it automatically → real business value.

That's the power of structured outputs. Not just 'an AI wrote something.' But 'an AI generated data that feeds into real systems that do real work.'

#### **DELIVERY TIPS**

These 5 real examples are the heart of this section—make them vivid. • Students know GitHub Copilot and LinkedIn—use that familiarity. • Each example shows: problem solved, business impact. Be concrete with numbers when possible. • The pattern at the end ties them together: shows this isn't coincidence, it's systematic. • These examples also motivate the lab—'you're building the same things these products do.'

## 09. BUILDING AI PIPELINES

~5 minutes

### WHAT'S ON SCREEN

Pipeline diagram: Input → AI generates → Validate → Transform → Output to System • Each step shown with example • Tools and technologies mentioned

### SPEAKING SCRIPT

Now let's talk about pipeline design. How do you connect all this together into a production system?

A PIPELINE is a sequence of steps: data comes in → gets processed → validated → transformed → fed to another system or output.

#### SIMPLE PIPELINE EXAMPLE:

##### STEP 1: INPUT

Data source: customer support email  
Format: raw text

##### STEP 2: AI GENERATION

Prompt: 'Extract ticket data from this email into JSON schema...'■r/>LLM generates: {"issue\_type": "...", "priority": "...", "urgency": "..."}■r/>Format: JSON (unvalidated)

##### STEP 3: VALIDATION

Check: is JSON valid? Do fields match schema? Are data types correct?■r/>If valid → continue. If invalid → retry or escalate.

Format: JSON (validated)

##### STEP 4: TRANSFORMATION (Optional)

Transform validated JSON into another format if needed.■r/>Example: Convert from 'customer\_email\_format' to 'internal\_ticket\_format'■r/>Maybe combine with other data sources (customer history, account info)

Format: Internal format

##### STEP 5: OUTPUT TO SYSTEM

Feed transformed data to: database, CRM, ticketing system, email alert, whatever.■r/>Format: System-specific (SQL insert, API call, webhook, etc.)

Result: Ticket automatically created, routed, tracked.

#### WHY PIPELINES MATTER:

1. MODULARITY: Each step can be replaced. Change the LLM? Just swap step 2. Change where data goes? Just change step 5.■r/>
2. ERROR HANDLING: At each step, you can catch errors and handle them. If validation fails, retry. If transformation fails, log it. If output fails, alert someone.■r/>
3. MONITORING: You can measure: how many emails processed, how many failed validation, how many tickets created. Visibility.

4. SCALABILITY: Pipelines can process thousands per second if architected right. Not one email at a time, but batch processing.■r/>

#### REAL PIPELINE: LINKEDIN RESUME PROCESSING

STEP 1: INPUT■r/>Resume uploaded by user (PDF file)■r/>

STEP 2: CONVERSION■r/>PDF → text (OCR if scanned)■r/>

STEP 3: AI EXTRACTION■r/>LLM prompted: 'Extract resume data to schema...'■r/>Generates JSON

STEP 4: VALIDATION■r/>Check JSON validity, validate against schema■r/>If invalid: retry with refined prompt or manual review■r/>

STEP 5: ENRICHMENT

Combine extracted data with external data (salary data, industry benchmarks, skill mapping to LinkedIn's taxonomy)■r/>  
STEP 6: DATABASE INSERT■r/>Insert validated and enriched data into LinkedIn's database

#### STEP 7: PROFILE UPDATE

User's profile is auto-populated■r/>

#### STEP 8: MONITORING

Track: success rate, parsing errors, user corrections (users manually fix bad extractions—that feedback improves the model)

#### TOOLS FOR BUILDING PIPELINES:

Apache Airflow: Schedule and monitor complex workflows

Zapier/Make: Visual workflow builders

Prefect: Modern workflow orchestration

Custom code: Python, Node.js, etc. with libraries like requests (API calls), pydantic (validation)

#### IN TODAY'S LAB:

You'll build a simplified version of this: prompt for JSON → validate it → extract fields → transform it → do something with it. Same principles as production systems, just smaller scale.

#### ***DELIVERY TIPS***

Pipeline diagrams are powerful—students need to see the flow. • The LinkedIn example shows real-world complexity—shows they're learning things that actually matter. • Break down each step simply: Input → Process → Validate → Output. • Emphasize that this is modular—if one step fails, you don't restart everything. • Connect to familiar concepts: error handling, monitoring, scaling.



## 10. LAB EXERCISE OVERVIEW

~3 minutes

### WHAT'S ON SCREEN

Lab title: 'Build Your Resume Pipeline' • 6-step process with flowchart • Deliverables checklist • Emphasis on data validation and processing

### SPEAKING SCRIPT

Alright, lab time. You're going to build a real, functional pipeline. Not theoretical. A working system.■r/>

YOUR TASK: Take your production-ready resume prompt from Session 3. Modify it to output JSON. Generate the JSON. Validate it. Extract specific fields. Demonstrate the pipeline.

#### STEP 1 (3 min): MODIFY YOUR PROMPT

Take your Session 3 resume prompt. Add schema specification and JSON output instructions.

#### STEP 2 (3 min): GENERATE JSON

Run your modified prompt. Get JSON output for a resume.

#### STEP 3 (4 min): VALIDATE

Check: is the JSON valid? Do all fields match your schema? Use a JSON validator (you can paste into jsonlint.com or similar).

#### STEP 4 (2 min): IF INVALID, ITERATE

If JSON invalid: identify what's wrong, refine the prompt, regenerate.

#### STEP 5 (4 min): EXTRACT FIELDS

Write simple extraction commands (or pseudocode) showing: 'Extract name', 'Extract all skills', 'Extract experience years'.

#### STEP 6 (2 min): DEMONSTRATE PIPELINE

Show: Raw JSON → Validate ✓ → Extract fields → Use data (example: sort skills by frequency, count years of experience).

#### DELIVERABLES:

1. Your modified prompt (with JSON schema)
2. Generated JSON output
3. Validation report (valid or invalid + fixes applied)
4. Extracted fields (as JSON or table)
5. Pipeline diagram (can be simple: boxes and arrows showing flow)
6. Reflection: How would this scale to 1000 resumes? What would break?

### DELIVERY TIPS

Step-by-step makes it manageable. • Students build something they can actually show others. • The reflection question bridges to production thinking. • Emphasize: this is now real infrastructure, not just chatting with an AI.

## 11. KEY TAKEAWAYS FROM SESSION 4

~5 minutes

### **WHAT'S ON SCREEN**

6 Core Principles • Structured outputs, JSON, Schema, Validation, Pipelines, Real-world applications

### **SPEAKING SCRIPT**

#### **Takeaway One: Structured Output Connects AI to Real Systems**

Unstructured text is for humans. Structured data is for systems. When you want AI to feed into workflows, you need structured output.

#### **Takeaway Two: JSON is the Standard**

Know JSON. Understand it. It's everywhere. API-first world speaks JSON.

#### **Takeaway Three: Schema is Your Blueprint**

When prompting for structured output, define the schema explicitly. Every field, data type, constraint. The more precise, the fewer errors.

#### **Takeaway Four: Validation is Non-Negotiable**

Generated output will be invalid sometimes. Plan for it. Validate every output before processing. It's not extra complexity. It's essential.

#### **Takeaway Five: Pipelines Scale What Works**

Single prompt → single JSON works fine at small scale. But pipelines—with validation, transformation, monitoring—scale to thousands or millions.

#### **Takeaway Six: This is How Real Products Work**

GitHub Copilot, ChatGPT Code Interpreter, LinkedIn, customer service bots—all use these principles. You're learning real infrastructure design.

### **DELIVERY TIPS**

These six principles are immediately applicable. • Students will remember these and apply them when they build their own systems. • Emphasize that they've now learned real, professional techniques—not textbook theory.

## 12. COMING UP NEXT

~2 minutes

### **WHAT'S ON SCREEN**

Dark blue background • Title: 'Session 5: Advanced Retrieval & RAG' • Preview bullets • Closing remarks

### **SPEAKING SCRIPT**

Session 5 is where things get even more powerful. Structured outputs are about formatting. Retrieval-Augmented Generation is about grounding.

Here's the problem we're solving: LLMs can hallucinate. They can confidently state false information because they're pattern-matchers, not fact-checkers.

Solution? RAG: Retrieval-Augmented Generation. You give the model real documents to reference. 'Answer this question BASED ON this document.' Suddenly it's not hallucinating. It's citing sources.

LinkedIn uses this to match job postings to candidates. Banks use it for compliance. Legal firms use it to cite case law. If you want AI that's trustworthy and verifiable, you need RAG.

In Session 5, you'll learn: how to build a document retrieval system, how to prompt the model to use retrieved documents, how to cite sources. And in the lab, you'll build a simple retrieval system for your resume—so when someone asks 'What are Priya's skills?' your system retrieves the skills section and answers based on that.

Thank you all for today. Structured outputs are powerful. Master this and you can build real systems. See you in Session 5.

### **DELIVERY TIPS**

Keep this brief and energizing. • Session 5 connects naturally: now that outputs are structured, make them factual and grounded. • End with impact: 'You're learning real, professional AI infrastructure.'